PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL — LABORATÓRIO DE REDES DE COMPUTADORES

INTERCEPTAÇÃO E DEFESA DE UM PROTOCOLO DE APLICAÇÃO

Fase 2: interceptação ativa (Man-in-the-Middle)

Bernardo Garcia Gabriel Vieira Guilherme Cavazzotto — Porto Alegre, 2026

Objetivo

Posicionar um proxy entre o cliente e o servidor do protocolo KVP e mostrar que, sem integridade nem autenticação, o conteúdo pode ser lido e alterado em trânsito sem que as pontas percebam. O proxy atende várias sessões ao mesmo tempo e resiste a desconexões e mensagens malformadas.

Arquitetura do proxy MITM

O proxy (`proxy/proxy_mitm.py`) aceita a conexão do cliente, abre a conexão ao servidor real e faz o relay das mensagens nos dois sentidos. O redirecionamento é por configuração: o cliente aponta o `--host` para o proxy em vez do servidor, sem saber que fala com um intermediário.

Para cada mensagem (uma linha do KVP), o proxy inspeciona o conteúdo e aplica regras de modificação antes de repassar. As duas regras usadas na demonstração:

- **C → S:** troca o valor de `SET destino ...` por `conta-do-atacante` (adultera um comando que vai ao servidor).
- **S → C:** troca a resposta `VALUE saldo ...` por `VALUE saldo 999999` (adultera uma resposta que volta ao cliente).

Concorrência e robustez

O proxy usa uma thread por sessão e, dentro de cada sessão, duas threads de relay (uma por sentido), pois cada sentido fica bloqueado em leitura e não pode travar o outro. O estado compartilhado (contador de sessões, estatísticas e o log) é protegido por locks; as regras de modificação são só leitura. Cada sessão é isolada: uma desconexão ou uma linha malformada afeta apenas a própria sessão. O detalhamento está em `docs/CONCORRENCIA.md`.

Ambiente de teste

A demonstração rodou em **nós isolados**, não em loopback. Três contêineres com IPs distintos numa rede bridge `10.10.0.0/24`: cliente (`10.10.0.30`), proxy (`10.10.0.20`) e servidor (`10.10.0.10`). O tráfego entre eles passa por Ethernet real. O mesmo código roda igual em três

VMs separadas. Toda a evidência a seguir vem de **uma única execução** (`deploy/run_demo.sh`), capturada nos dois trechos ao mesmo tempo.

Evidência da modificação

Capturamos os dois trechos da mesma sessão. A Tabela 1 compara, nos mesmos pacotes, o que passou de cada lado do proxy.

Tabela 1 – O mesmo pacote nos dois trechos: o proxy altera o conteúdo.

Pacote	Trecho cliente↔proxy (.30↔.20)	Trecho proxy↔servidor (.20↔.10)
#10 (C→S)	`SET destino conta-do-bernardo`	`SET destino conta-do-atacante`
#20 (S→C)	`VALUE saldo 999999`	`VALUE saldo 100`

Fonte: os autores (2026).

As Figuras 1 e 2 mostram, no Wireshark, a mesma sessão reconstruída (*Follow TCP Stream*) nos dois trechos. No trecho do cliente aparece o que o cliente enviou e recebeu; no trecho do servidor, o que o servidor recebeu e enviou. A diferença entre as duas é a adulteração feita pelo proxy.

Figura 1 – Trecho cliente↔proxy: o cliente envia `conta-do-bernardo` e recebe `saldo 999999`.

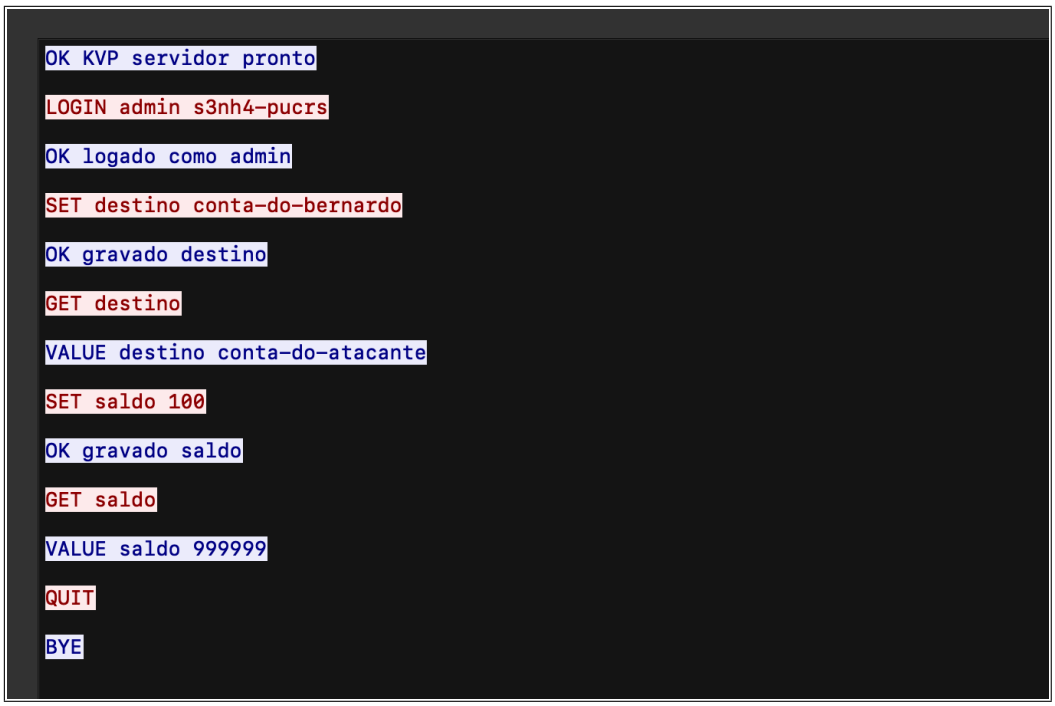
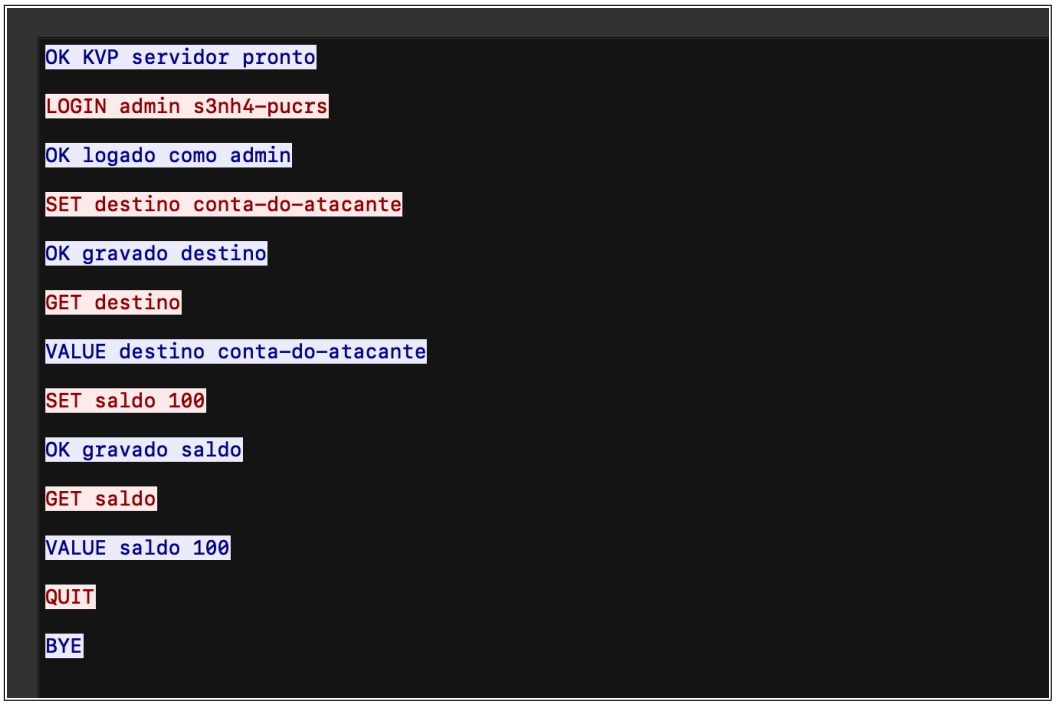


Figura 2 – Trecho proxy↔servidor: o servidor recebe `conta-do-atacante` e envia `saldo 100`.



O log do proxy registra a leitura e a troca (sessão do cenário principal):

```
C->S ORIGINAL: SET destino conta-do-bernardo
C->S ALTERADA: SET destino conta-do-atacante   (regra_redireciona_destino)
S->C ORIGINAL: VALUE saldo 100
S->C ALTERADA: VALUE saldo 999999              (regra_infla_saldo)
```

O servidor aceitou o comando adulterado como legítimo (gravou `conta-do-atacante`) e o cliente aceitou a resposta adulterada como legítima (leu `saldo 999999`). Nenhuma das pontas tem como detectar a troca.

Concorrência e robustez (evidência)

No mesmo experimento, três clientes passaram pelo proxy ao mesmo tempo. O log mostra as sessões abertas simultaneamente antes de qualquer uma encerrar:

```
[sessao 4] nova conexao de 10.10.0.30:41926
[sessao 5] nova conexao de 10.10.0.30:41916
[sessao 6] nova conexao de 10.10.0.30:41938
[sessao 4] sessao encerrada
[sessao 5] sessao encerrada
[sessao 6] sessao encerrada
```

Para a robustez, um cliente que enviou bytes inválidos e desconectou no meio rodou em paralelo com um cliente normal; o normal concluiu sem ser afetado (ver `fase2_robustez_cliente.txt` em `docs/capturas`).

Conclusão

O proxy MITM lê e altera as mensagens do KVP em trânsito, e as duas pontas aceitam o conteúdo adulterado. Isso confirma a quebra de integridade e de autenticação de um protocolo sem proteção. Na Fase 3, o canal receberá TLS (para cifrar e garantir integridade) e autenticação do servidor por certificado, o que impede o proxy de se passar pelo servidor e de alterar as mensagens sem ser detectado.